

Contribution Log

Kashaf Fatima · 84381

CEREBRO and APOLLO-M · BS Computer Science · Bahria University, Karachi Campus
Supervisor: Mr. Saghir Ahmed

About this log

This is my individual contribution record across both systems of the project: **CEREBRO**, the threat and misinformation intelligence platform, and **APOLLO-M**, the community-instability forecasting framework that consumes it. The two share a deployed PostgreSQL instance and a common design principle: deterministic code and trained models produce every number, and a language model is used only to explain them.

Entries record work I carried out personally. Where I corrected, integrated or extended a component, the entry describes that correction or integration. Estimated effort is per session. Code lives in the GitHub repositories; this is the process and contribution record.

Part I — CEREBRO

Areas of contribution

Area	What I contributed
Frontend (React / TypeScript)	The entire user interface: dashboard, the News, Email and Network scanners, the authentication screens, and the animated global threat map on the login background. Rewired authentication to the backend, added mobile-safe token sessions, made fact-check sources clickable, and cleared the type-checking errors.
Backend (FastAPI / Python)	The async database access layer — connection pool, per-tenant row-level security, realtime notifications — the authentication system with bcrypt password hashing and JWT sessions, request persistence, and the wiring of feature extraction into the API.
Middleware & APIs	All /v1 endpoints: auth, metrics, flows, a WebSocket stream, and news, network and email analysis, plus forecasting. CORS handling, the cross-site cookie and bearer-token logic, a named-query loader and safe error handling.
Database	Deployed the PostgreSQL schema with pgvector, enforced per-tenant isolation, and made every analysis persist so the data is collectable — the dataset that feeds the Apollo model.
Machine learning & data pipelines	Trained a phishing classifier on the Apache SpamAssassin corpus of roughly 7,400 real emails, reaching a held-out F1 of about 0.97 (AUC-ROC 0.996). Built the unsupervised anomaly pipeline and the forecasting scaffold with a rolling-origin backtest over seven months of real time-series data, wrote the dataset fetchers, and integrated the Google Fact Check Tools API for citable sources.
Retrieval & explanation layer	Added a Claude provider to the language-model chain with structured output via forced tool use, and built a 17-document evidence corpus with source credibility weights. Refutation of representative false claims went from 0 of 10 to 9 of 10, each with citations a reader can open.

Deployment / DevOps	Dockerised the backend and deployed it to Render with managed PostgreSQL, deployed the frontend to Vercel, set up the repository and CI, ran the full stack locally in Docker, and resolved the cross-origin, cookie and mobile issues so it works publicly on every device.
---------------------	--

Dated activity log

27 Jul 2026 · Est. effort: ~1.5 h
Code review & baseline assessment
ACTIVITIES
- Reviewed the CEREBRO codebase — email forensics, security and cryptography, retrieval verification, frontend. - Ran the backend test suite and audited the core modules for correctness. - Identified the main gap: the FastAPI-to-database layer and the /v1 routers were not built.
Outcome: Baseline established; the API-to-database wiring pinned as the priority.
27 Jul 2026 · Est. effort: ~3 h
Completing the API ↔ database layer
ACTIVITIES
- Built the async database layer with asyncpg: connection pool, per-request row-level security tenant scoping, and a LISTEN/NOTIFY realtime fan-out. - Wrote a named-query loader and a JWT/cookie authentication dependency. - Implemented the auth, metrics, flows, WebSocket stream, news, network and forecast routers and wired them into the application.
Outcome: The documented blocker resolved; backend test suite green at 67 of 67.
27 Jul 2026 · Est. effort: ~1 h
Frontend correctness fixes
ACTIVITIES
- Renamed the mis-named service and repointed calls to the /v1 API. - Removed fabricated dashboard statistics that had survived from the previous version. - Fixed two pre-existing TypeScript errors so the frontend type-checks cleanly.
Outcome: Frontend type-checks pass; the UI calls the real backend consistently.

27–28 Jul 2026 · Est. effort: ~4 h

Trained models, datasets & forecasting

ACTIVITIES

- Wrote the phishing-classifier wrapper, a training script and a dataset fetcher.
- Trained the classifier on roughly 7,400 real emails from the Apache SpamAssassin corpus — held-out F1 about 0.97, AUC-ROC 0.996.
- Added a loader for the anomaly detector and wrote the forecasting scaffold with a rolling-origin backtest over seven months of real time-series data.

Outcome: Email model trained and verified on real data; anomaly and forecasting pipelines runnable.

28 Jul 2026 · Est. effort: ~1 h

Gmail OAuth server-side exchange

ACTIVITIES

- Implemented the authorization-code to token exchange endpoint.
- Encrypted the returned tokens at rest with AES-256-GCM bound to the user.

Outcome: The token flow moved off the browser and secured server-side.

28 Jul 2026 · Est. effort: ~2 h

Deployment preparation

ACTIVITIES

- Wrote a Render blueprint for one-click backend and PostgreSQL, a slim runtime image and a Vercel configuration.
- Added a deployment runbook, a cross-site cookie option, automatic schema application and a demo-user seed for fresh instances.

Outcome: Repository made deploy-ready for a free cloud stack.

30 Jul 2026 · Est. effort: ~2 h

Local Docker run & environment workarounds

ACTIVITIES

- Ran the full stack locally in Docker to verify it end to end.

- Worked around a network restriction preventing an image pull by building it from the base PostgreSQL image.
- Found and fixed two bugs that only surface on first boot: a missing dependency and an invalid demo-email domain.

Outcome: Whole platform confirmed running locally — login, email forensics and metrics working.

30–31 Jul 2026 · Est. effort: ~2 h

Public deployment

ACTIVITIES

- Initialised the repository, committed the work and pushed to GitHub.
- Deployed the backend to Render with managed PostgreSQL and the frontend to Vercel.
- Fixed cross-origin access and the cross-site session cookie, and verified end-to-end login from the live frontend.

Outcome: CEREBRO live on the public internet — frontend on Vercel, backend and database on Render.

31 Jul 2026 · Est. effort: ~2 h

Frontend ↔ backend authentication integration

ACTIVITIES

- Found the frontend still using leftover third-party authentication, disconnected from the backend.
- Replaced it with backend login, register and session handling, and added a registration endpoint.

Outcome: One coherent authentication system across the application.

31 Jul 2026 · Est. effort: ~3 h

Feature completion & fixes

ACTIVITIES

- Built the animated global threat map for the login screen.
- Made the news scanner return varied, defensible scores through a transparent linguistic heuristic, and integrated the Google Fact Check Tools API for citable sources with clickable links.
- Fixed the network scanner with transparent rule-based triage over labelled sample flows, and replaced the broken mail panel with a working email forensics scanner.

Outcome: All three detection modules working end to end with explainable output.

31 Jul 2026 · Est. effort: ~1.5 h

Persistence — data collection for Apollo

ACTIVITIES

- Made every email, news and network analysis write to PostgreSQL.
- Confirmed the dashboard metrics populate and the data is queryable.

Outcome: CEREBRO collects a real, queryable dataset for the parent project.

31 Jul 2026 · Est. effort: ~1 h

Mobile / iOS authentication fix

ACTIVITIES

- Diagnosed and fixed the session behaviour that prevented sign-in on mobile browsers.

Outcome: The app works on phones, in-app browsers and desktops alike.

11 Aug 2026 · Est. effort: ~2 h

Claude provider & evidence corpus

ACTIVITIES

- Added a Claude provider to the language-model chain using the Messages API over the existing HTTP client, so no new dependency entered the image, and implemented structured output through forced tool use — without which claim extraction silently degraded to a sentence split.
- Built a 17-document evidence corpus with per-publisher credibility weights, and found and fixed a silent join defect that had been giving the strongest sources a default weight.
- Measured refutation of representative false claims: 0 of 10 before, 9 of 10 after, each with citations a reader can open.

Outcome: Misinformation verdicts now cite retrieved evidence and explain themselves.

Part II — APOLLO-M

Areas of contribution

Area	What I contributed
------	--------------------

Data integrity & pipeline correction	Audited the pipeline end to end and found six defects that had already produced published numbers: comment text assigned to communities at random with a toxicity score generated by <code>np.random</code> rather than by a model; a loader reading a fixed prefix of a name-sorted file, so analysis covered only communities beginning with 'A' — 33 of 831 eligible; polarisation and echo-chamber computed once globally and copied to every community; a metrics script silently overwriting the health report using a different formula; and three headline accuracies that were hardcoded string literals. Corrected each, and replaced the fabricated figures with an explicit unevaluated status and the reason.
Validation framework & the instability score	Built the ground-truth validation the project had no equivalent of: a declared simulation of 60 communities over 120 days in which 15 receive a recorded rising trend the pipeline never sees, and a harness reporting ROC-AUC, precision and recall at k, and forecast-direction agreement. This produced the project's central finding — the Community Health Index ranked communities less accurately (0.575) than one of its own inputs (0.649) — and I designed the trend-aware instability score in response, which reaches 1.000 on the same task. The forecaster was measured independently at 15 of 15 recall with a slope ROC-AUC of 0.732.
Real Reddit ingestion	Built live ingestion against Arctic Shift, which requires no credentials and so bypasses the account-age and karma gate on Reddit's own API. Diagnosed two API behaviours that had stalled collection: results are returned newest-first, so pagination must walk backwards, and date-string cursors resolve only to a day and never advance. Added per-day sampling for a usable daily series, unattended collection that merges by comment id, and retry handling. Collected 108,139 real comments across 24 communities and 64,416 authors over 40 days.
Backend & API (FastAPI)	Built the serving layer and corrected its faults: one endpoint read a stale artifact and could only fail, another returned all 300 forecast rows for every request when its lookup missed, silently answering with another community's data. Added a public metrics endpoint in Prometheus exposition format, since the only scrape target had been a local exporter. Anchored file paths to the repository root and made CORS configurable.
Database & shared instance	Designed the deployed persistence: Apollo and CEREBRO share a single PostgreSQL instance with schema isolation — the shared-database integration the project describes, actually deployed and verified. Added connection-string support for managed providers, idempotent migrations, and persistence for the instability score, structural risk and recommended action, which the pipeline computed but which had no columns and so never reached the API.
Dashboard (Streamlit)	Built the console: login with role-based accounts and Google OIDC, seven pages, a self-driving live feed that works for every viewer rather than only the operator, a monitoring page that renders metrics instead of linking to localhost, a real-Reddit page with a live fetch that calls the API when a visitor presses it, and a dataset switch so the benchmark and the real corpus can each be inspected without one overwriting the other.
Security	Found the JWT signing key hardcoded in a repository that had just been made public — enough for anyone reading it to mint a valid admin token — and moved it to the environment with a random per-process fallback, verifying that a token forged with the old key is now rejected. Replaced unsalted SHA-256 password hashing with PBKDF2-HMAC-SHA256, and wired the CORS setting, which had been inert.
Deployment / DevOps	Deployed the API and managed PostgreSQL on Render and the dashboard on Streamlit Community Cloud, diagnosing each failure in turn: an SSL mode incompatible with the provider's internal network, a requirements file the platform does not read — it was installing PyTorch into a dashboard that never imports it — a duplicate widget identifier, and a missing dependency. Initialised the repository and reduced it from 526 MB to 370 KB by excluding corpora that regenerate.

Monitoring	Prometheus exporter, provisioned Grafana dashboard, metrics for the trend-aware signal, and a widened default time range so the series is visible from a cold start.
Documentation	Nine architecture and design figures generated from the code itself so they cannot drift from it, the corrected technical report with a data-provenance section, the FYP Mid Evaluation Report on the university template, and this log.

Dated activity log

9 Aug 2026 · Est. effort: ~2 h

Forecasting defect and monitoring stack

ACTIVITIES

- Found the forecaster returning three identical quantiles, so every confidence band had zero width; corrected the prediction mode and horizon indexing and verified the bands widen with horizon as they should.
- Stood up the Prometheus exporter, a provisioned Grafana dashboard and the metric series behind them.

Outcome: Forecast intervals became meaningful; monitoring operational.

9–10 Aug 2026 · Est. effort: ~4 h

Running the pipeline; two defects found

ACTIVITIES

- Ran the full pipeline rather than reading it, and found the Community Health Index was zero for every community — a scaling error multiplying an already-bounded penalty by 100.
- Found sparse communities producing coin-flip toxicity means and restored a minimum-comments filter.
- Built the Streamlit console: login, navigation, per-community drill-down, forecasts, clusters and a live processing view.

Outcome: The health index varied correctly for the first time; dashboard usable end to end.

10 Aug 2026 · Est. effort: ~3 h

Backend, database and the shared instance

ACTIVITIES

- Built the FastAPI service with JWT authentication and role-based access.
- Created the database schema and loader, and established the shared PostgreSQL instance serving both Apollo and CEREBRO with schema isolation.

- Added the provider-switchable explanation layer, keeping the language model to narration only so no score or decision originates from it.

Outcome: Results served from a database rather than read from files.

11 Aug 2026 · Est. effort: ~2 h

Data integrity audit

ACTIVITIES

- Traced the corpus build and found comment text assigned to communities at random, and the toxicity score generated by np.random.uniform rather than by any model.
- Found the loader reading a 50,000-row prefix of a name-sorted file, so every published figure described communities beginning with 'A' — 33 of 831 eligible.
- Replaced it with selection by comment volume, and found that the larger, correct sample collapsed the apparent toxicity spread from 0.67–0.83 to 0.543–0.559, showing the original differences had been sampling noise.

Outcome: The dataset's limits established and documented rather than inherited.

11 Aug 2026 · Est. effort: ~3 h

Ground-truth validation and the instability score

ACTIVITIES

- Built a declared simulation with recorded ground truth and a validation harness measuring detection and forecasting against it.
- Measured the Community Health Index at ROC-AUC 0.575 — below raw toxicity at 0.649 — and diagnosed the cause as an index composed only of present-tense quantities, which cannot rank communities by rate of change.
- Designed and implemented the trend-aware instability score; re-measured at 1.000, with the forecaster independently at 15 of 15 recall and slope ROC-AUC 0.732.

Outcome: A negative result about the project's own contribution, and its correction.

11 Aug 2026 · Est. effort: ~1.5 h

Fabricated metrics removed

ACTIVITIES

- Found three headline accuracies written as string literals — 99.07%, 99.77% and a training loss of 0.0077 — flowing into the dashboard, Grafana and the technical report as if they had been measured.
- Replaced them with an explicit unevaluated status and the reason for it.

- Found and stopped a metrics script silently overwriting the health report using a different formula from the pipeline's, which had made published figures depend on which script ran last.

Outcome: Every remaining figure in the project is reproducible from a script.

11 Aug 2026 · Est. effort: ~3 h

Security and cloud deployment

ACTIVITIES

- Found the JWT signing key hardcoded in a newly public repository and moved it to the environment; verified that a token forged with the old key is now rejected.
- Replaced unsalted SHA-256 password hashing with PBKDF2-HMAC-SHA256.
- Deployed the API and database to Render and the dashboard to Streamlit Cloud, resolving SSL-mode, requirements-file, widget-identifier and dependency failures in turn.

Outcome: Both systems publicly reachable; the credential vulnerability closed.

11 Aug 2026 · Est. effort: ~2 h

Integration of the remaining model layers

ACTIVITIES

- Recovered trained artifacts from an earlier archive that were absent from the working copy, and integrated the structural, misinformation and moderation layers into the pipeline so their outputs reach the database and the API.
- Reimplemented graph-model inference in plain PyTorch, since the module's dependency on torch_geometric — which is not installed — was the reason the trained model had never run.
- Measured the misinformation classifier as out-of-domain on short social text, flagging 99.5% of posts, and recorded the diagnostic rather than publishing a rate the data cannot support.

Outcome: Three previously unused layers now contribute columns the API serves.

11 Aug 2026 · Est. effort: ~3 h

Real Reddit ingestion

ACTIVITIES

- Built an Arctic Shift adapter requiring no credentials, diagnosing backwards pagination and day-resolution cursors as the causes of stalled collection — together these took collection from 98 to several thousand comments per community.
- Added per-day sampling, unattended collection that merges by comment id, and retries.

- Collected 108,139 real comments across 24 communities and 64,416 authors over 40 days, and added a dataset switch so the benchmark and the real corpus coexist.

Outcome: The system runs on genuine Reddit data, not only on a benchmark.

11 Aug 2026 · Est. effort: ~3 h

Figures, technical report and FYP report

ACTIVITIES

- Replaced three diagrams that showed unintegrated modules as active pipeline stages, generating nine figures from the code so they cannot drift from it.
- Corrected the technical report: fabricated metrics removed, a data-provenance section added, and a claim that tests existed withdrawn.
- Built the FYP Mid Evaluation Report on the university template — 45 headings, six tables, nine figures and twenty-two references.

Outcome: Documentation consistent with the system it describes.

Summary of my contribution

- Completed CEREBRO's missing API-to-database layer and made it a coherent, working system, then deployed the whole platform publicly.
- Trained a phishing classifier on roughly 7,400 real emails (F1 about 0.97) and built the anomaly and forecasting pipelines with a real-data backtest.
- Built CEREBRO's evidence corpus and Claude explanation layer: refutation of representative false claims went from 0 of 10 to 9 of 10, each with citations.
- Audited APOLLO-M end to end and found six defects that had produced published figures, including three accuracies that were hardcoded literals rather than measurements.
- Built the ground-truth validation framework, established that the project's own health index ranked communities less accurately (0.575) than one of its inputs (0.649), and designed the trend-aware instability score in response.
- Measured the forecaster independently: 15 of 15 recall on planted destabilising communities, slope ROC-AUC 0.732.
- Built credential-free Reddit ingestion and collected 108,139 real comments across 24 communities and 64,416 authors.
- Removed a hardcoded JWT signing key from a public repository and replaced unsalted password hashing with a real key-derivation function.
- Deployed both systems publicly on a shared PostgreSQL instance with schema isolation, with monitoring, and produced the figures, technical report, FYP report and this log.

Live systems. CEREBRO — cerebro-sandy-beta.vercel.app · API cerebro-api-nmah.onrender.com
APOLLO-M — apollo-m.streamlit.app · API apollo-api-tlm.onrender.com
Code — github.com/kashaffatimajaffrey-design/CEREBRO · github.com/kashaffatimajaffrey-design/apollo-m